

Root-me: ELF x86 — Basic | Полное прохождение

Что вы узнаете

К концу руководства вы научитесь:

- Проводить статический анализ бинарного файла
- Менять поток выполнения программы
- Изменять значения регистров во время выполнения в отладчике

Необходимые знания

Базовое понимание ассемблера x86, работы в терминале Linux и умение пользоваться отладчиком GDB.

Шаг 1. Подготовка файла

Файл приходит в архиве .zip. Распаковываем его:

```
unzip ch2.zipfile ch2.bin
```

Результат команды file:

```
ch2.bin: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), statically linked,  
for GNU/Linux 2.6.8, with debug_info, not stripped
```

Бинарник **статически линкован**, 32-битный, с отладочной информацией и **не застриппован** — это сильно упрощает анализ.

Шаг 2. Первичный запуск

Запускаем программу:

```
./ch2.bin
```

Программа выводит приветствие на французском/испанском и просит ввести username:

```
#####  
##      Bienvennue dans ce challenge de cracking      ##  
#####  
  
username: test  
Bad username
```

При любом неправильном вводе программа выводит Bad username и завершается.

Шаг 3. Анализ в GDB (Intel-синтаксис)

Открываем файл в GDB и настраиваем удобный Intel-синтаксис:

```
gdb ch2.bin
```

```
(gdb) set disassembly-flavor intel
```

```
(gdb) layout prev
```

Основная функция `main` выглядит так (дизассемблированный код):

```
0x08048309 <+0>:  lea    ecx,[esp+0x4]
0x0804830d <+4>:  and    esp,0xffffffff
0x08048310 <+7>:  push   DWORD PTR [ecx-0x4]
0x08048313 <+10>: push   ebp
0x08048314 <+11>: mov    ebp,esp
0x08048316 <+13>: push   ecx
0x08048317 <+14>: sub    esp,0x24
0x0804831a <+17>: mov    DWORD PTR [ebp-0xc],0x80a6b19 ; указатель на username
0x08048321 <+24>: mov    DWORD PTR [ebp-0x10],0x80a6b1e ; указатель на password
0x08048328 <+31>: mov    DWORD PTR [esp],0x80a6b2c
0x0804832f <+38>: call   0x8048de0 <puts> ; вывод приветствия
...
0x0804835e <+85>: call   0x804826a <getString> ; ввод username
0x08048366 <+93>: mov    eax,DWORD PTR [ebp-0xc]
0x08048373 <+106>: call   0x80502f0 <strcmp> ; сравнение username
0x08048378 <+111>: test   eax,eax
0x0804837a <+113>: jne    0x80483d0 <main+199> ; если не равно → Bad username
...
0x08048383 <+122>: call   0x804826a <getString> ; ввод password
0x080483a3 <+154>: call   0x80502f0 <strcmp> ; сравнение password
0x080483a8 <+159>: test   eax,eax
0x080483aa <+161>: jne    0x80483c2 <main+185> ; если не равно → ошибка
...
0x080483bb <+178>: call   0x8048db0 <printf> ; успех + флаг
```

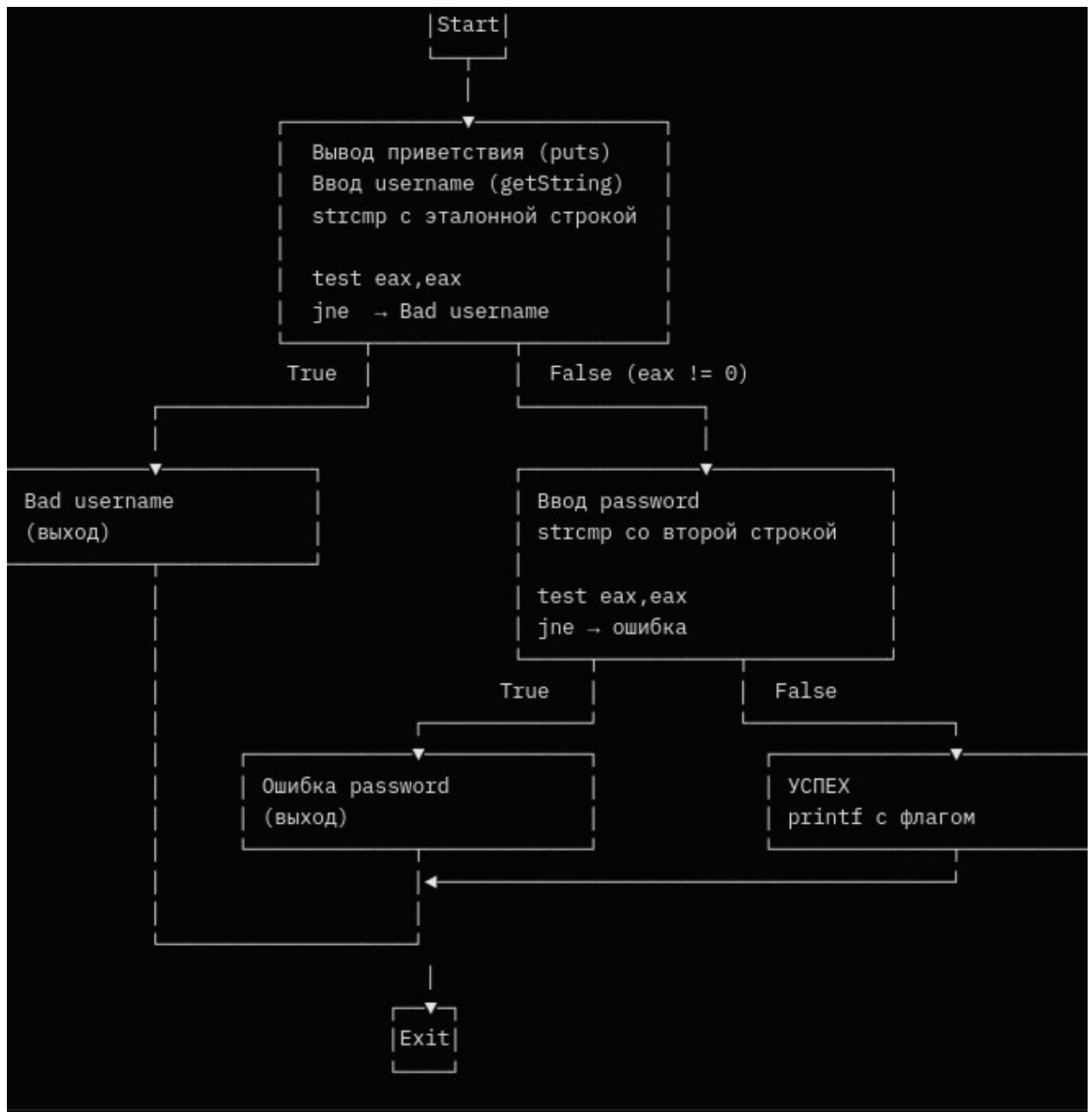
Шаг 4. Логика работы программы (схема)

Программа выполняет **две проверки** через `strcmp`:

1. Сравнивает введённый `username` с жёстко заданной строкой.
2. Если первая проверка пройдена — просит `password` и сравнивает его со второй строкой.

Если хотя бы одна проверка не пройдена — выводится ошибка и программа завершается.

Вот **схема потока выполнения** программы:



Шаг 5. Обход проверок в GDB (динамический анализ)

Самый простой способ — не подбирать строки, а **принудительно обойти обе проверки**, установив регистр `eax = 0` после каждого `strcmp`.

Обход первой проверки (username)

```
(gdb) break *0x08048378 # точка останова на test eax, eax
```

```
(gdb) run
```

```
(gdb) set $eax = 0 # принудительно говорим, что строки совпали
```

```
(gdb) ni
```

Теперь программа считает, что username правильный, и переходит к запросу пароля.

Обход второй проверки (password)

```
(gdb) break *0x080483a8      # точка останова на втором test eax,eax
```

```
(gdb) continue
```

```
(gdb) set $eax = 0
```

```
(gdb) ni
```

После этого программа доходит до финального printf и выводит флаг.

Флаг

```
Bien joue, vous pouvez valider l'epreuve avec le mot de passe : 987654321 !
```

Флаг: 987654321

Итог

В этом челлендже не нужно было угадывать или декодировать строки. Достаточно было понять логику двух последовательных проверок strcmp и научиться манипулировать регистром eax в отладчике, чтобы заставить программу считать обе проверки пройденными.

Это классический пример динамического обхода условий в crackme.